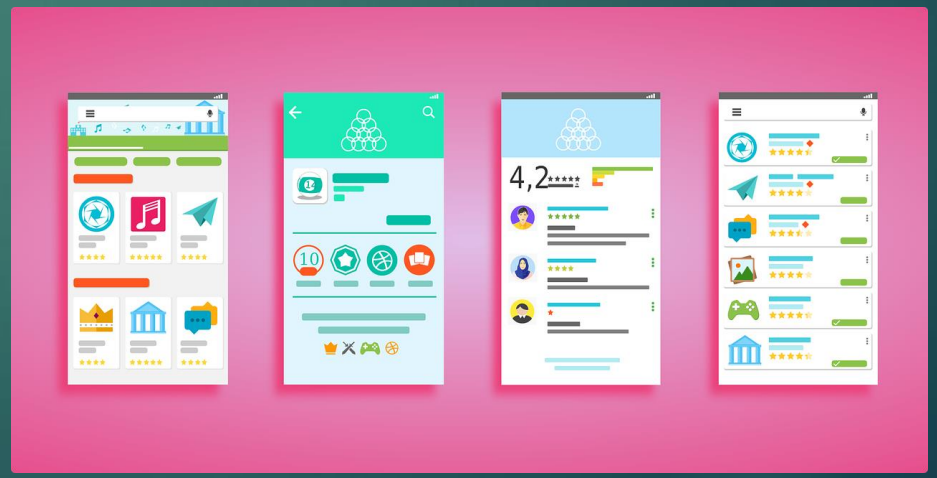
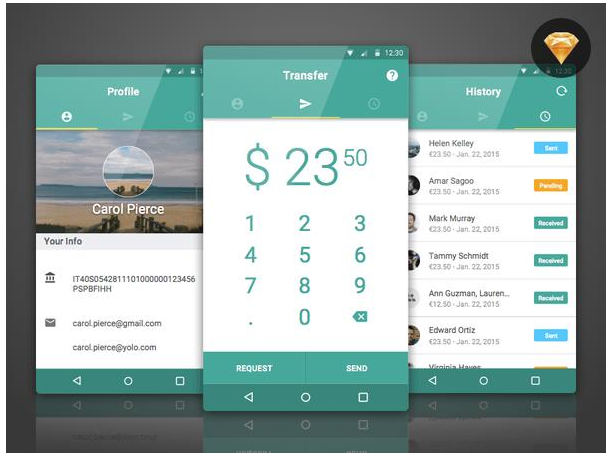
4. Android: User Interface

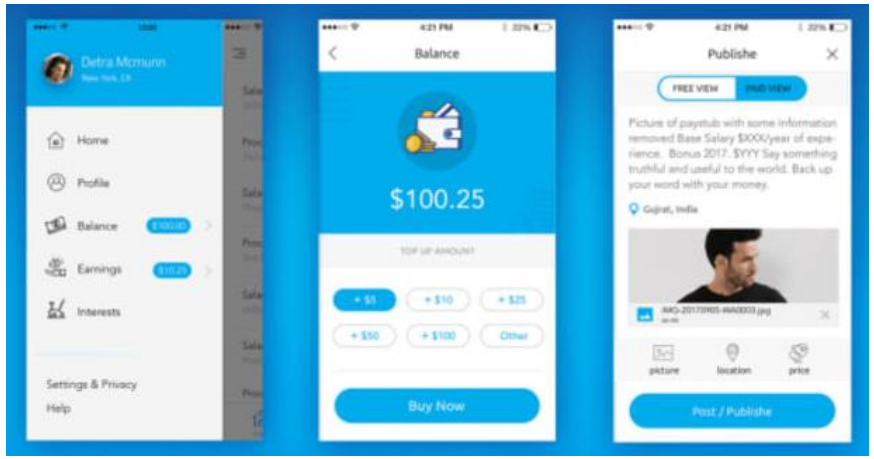


LEIM – DAM

User Interface (UI)

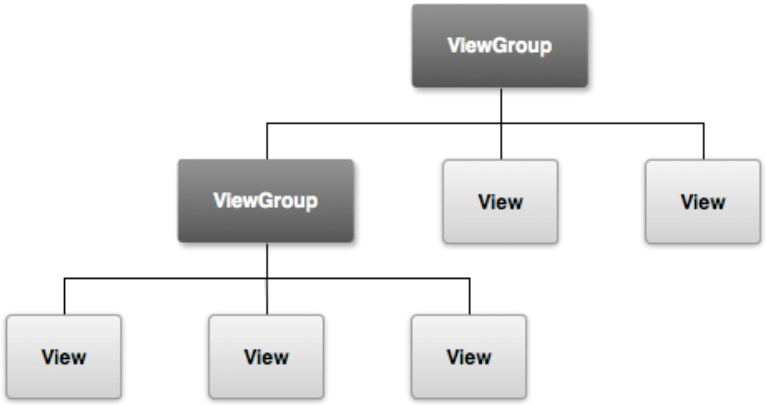
- App's user interface is what the user see and interact with
  - Android have many UI components
    - Layout, widgets
  - Android have UI modules for special interfaces
    - dialogs, notifications, menus, …



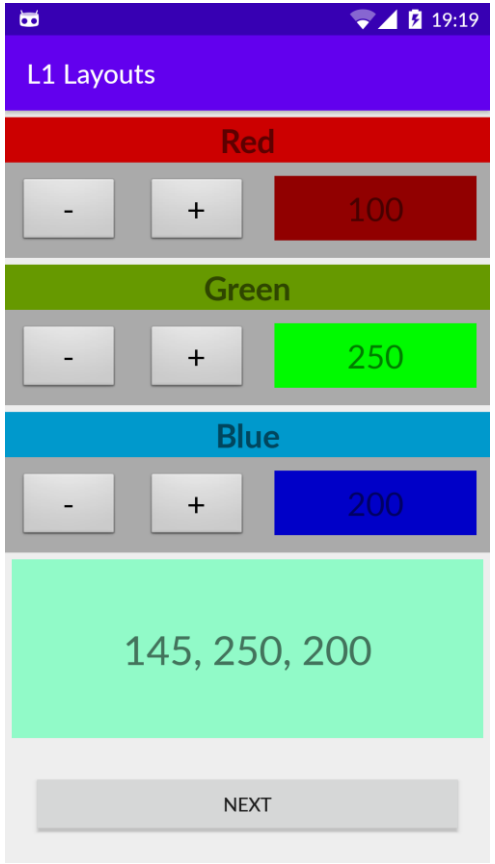


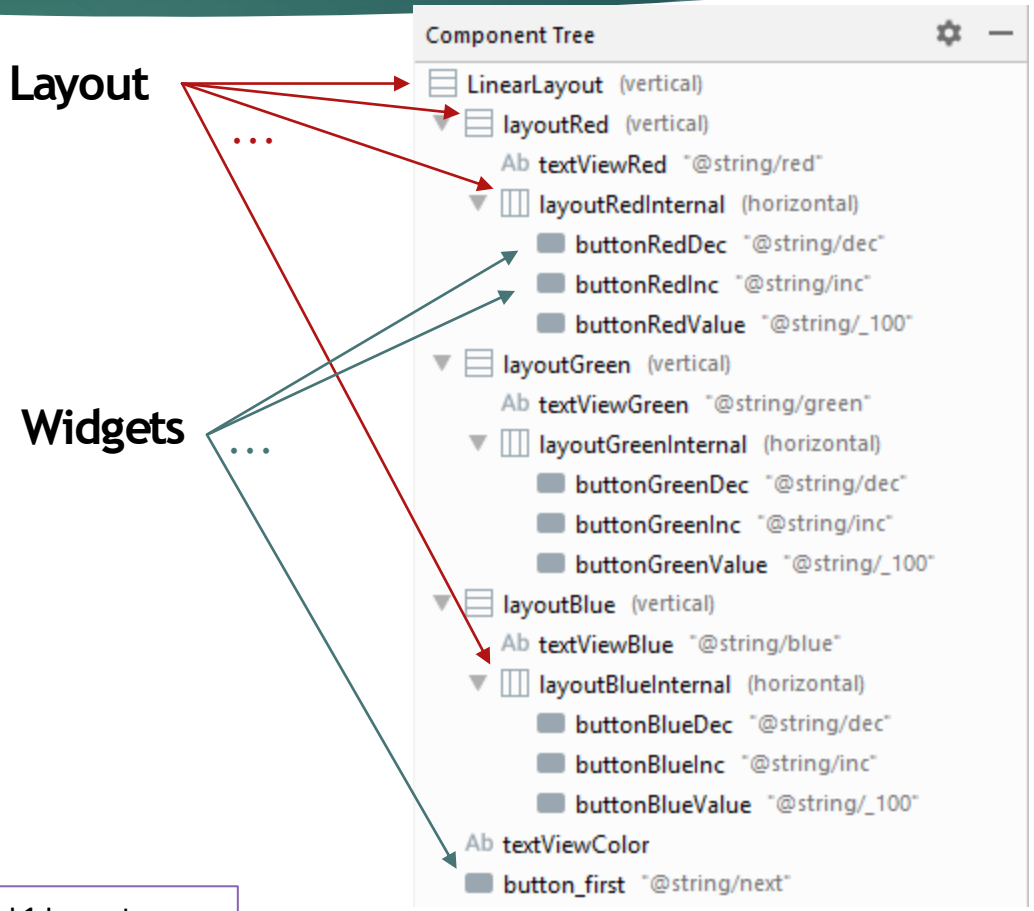
User Interface (UI)

- UI elements are based on:
  - **View** Class – is/describes one UI visual element
    - Base class for "widgets". Ex: TextView, ImageView, Button, RatioButton, …
    - Usually draws something the user can see and interact with
  - **ViewGroup** class – an element to contain other elements
    - May contain Views and ViewGroups
    - Base class for "layouts"
      - Ex: ConstraintLayout, LinearLayout, FrameLayout, GridLayout, …
    - It is a View sub-class
    - Should be the root element in a Layout file



UI: ViewGroups and Views





L1 Layouts app

User interface statically defined in XML

- The user interface can be **declared in XML files**
  - These files are called **layout files** (but they just contain elements)
  - These file describe the UI elements that will be shown
    - They should contain one element, usually a ViewGroup
      - These root elements usually contain other elements
      - Forming a tree of elements, called the View Hierarchy
  - **activity_main.xml** is the usual name for the first activity layout file
  - Layout in activities
    - To set a layout file in the current activity, use: Activity.**setContentView** in onCreate
    - The rendering starts from the root node and goes all the way down the tree
      - If Views overlap, the last one to be rendered in the one visible

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Layout: activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <LinearLayout
        android:id="@+id/layoutRed"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="5dp"
        android:background="@android:color/darker_gray"
        android:orientation="vertical">

        <TextView
            android:id="@+id/textViewRed"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@android:color/holo_red_dark"
            android:text="@string/red"
            android:textAlignment="center"
            android:textSize="24sp"
            android:textStyle="bold" />
        ...
```

Each XML element type have an Java class with the same name and that for each element it is instantiated one instance of that class:

<TexView> -> **TextView**
<RelativeLayout> -> **RelativeLayout**

When Android loads a layout (in XML) it creates and initialize the objects described in the XML elements and with the properties defined

Initial content of activity_main.xml of L1 Layouts app

Resource attributes

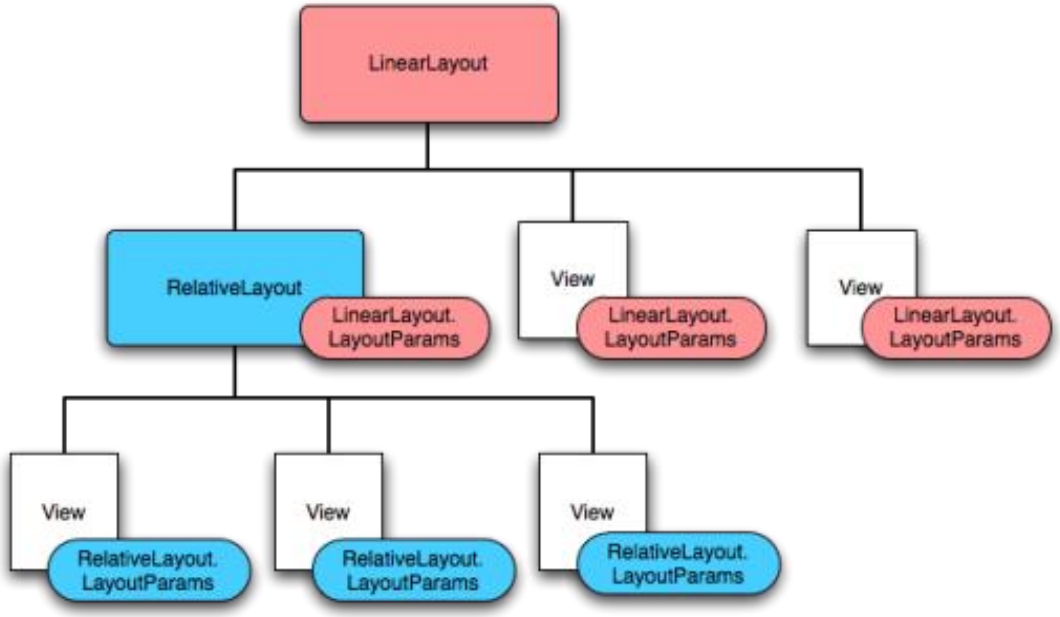
- View or ViewGroup attributes
  - Specific attributes
    - textSize - <TextView>
  - Inherited
    - From the View root (e.g., ID attribute)
  - LayoutParams
    - Attributes that describe the orientation of the View object in the layout

Resource ID attribute

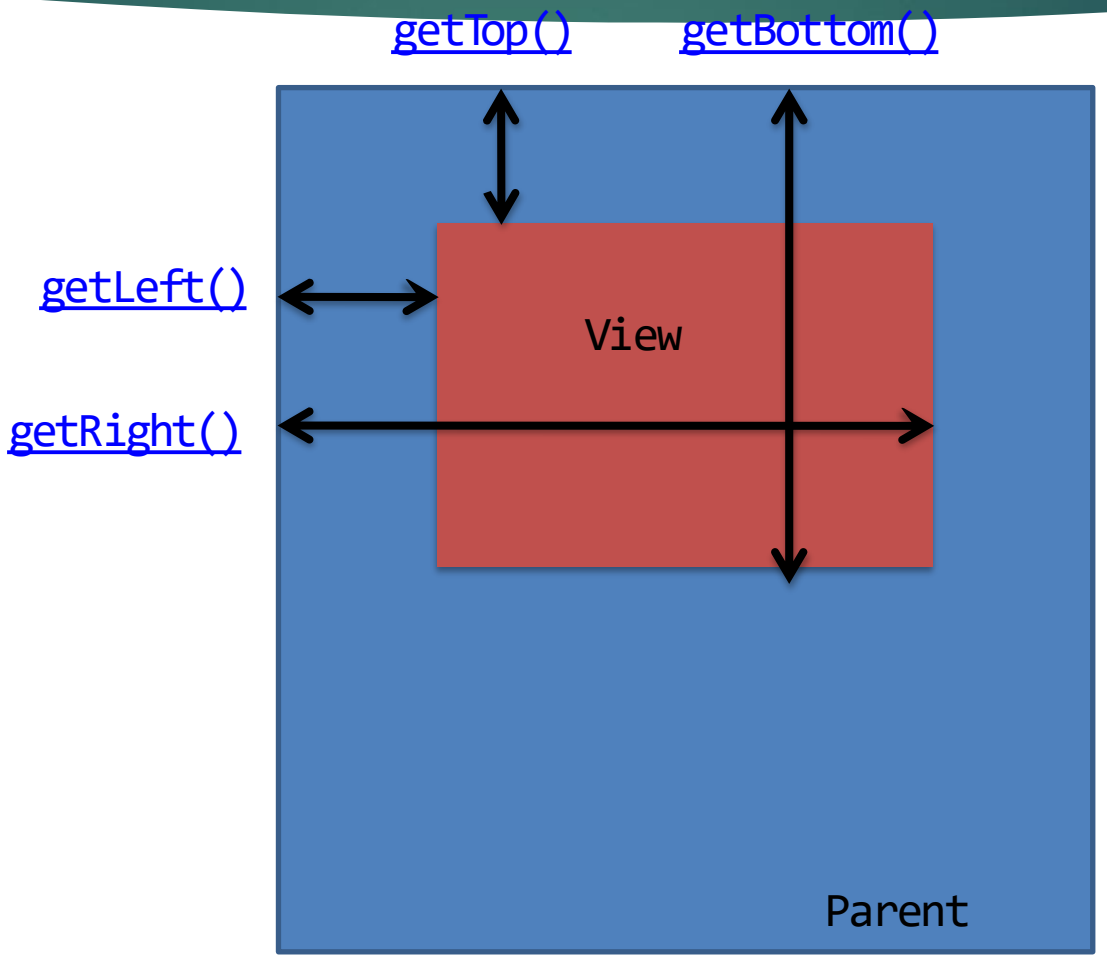
- ID attribute associated with each View object to uniquely identify it
- When the application is compiled
  - string XML is converted in an integer
- Sintaxe:
  - `android:id="@+id/elemIdentifier"`
  - The @ symbol indicates that the XML parser must expand the rest of the string ID and identify it as an asset ID.
  - The + symbol means that we have a new element that must be added to the resources (R.java)

Layout Parameters

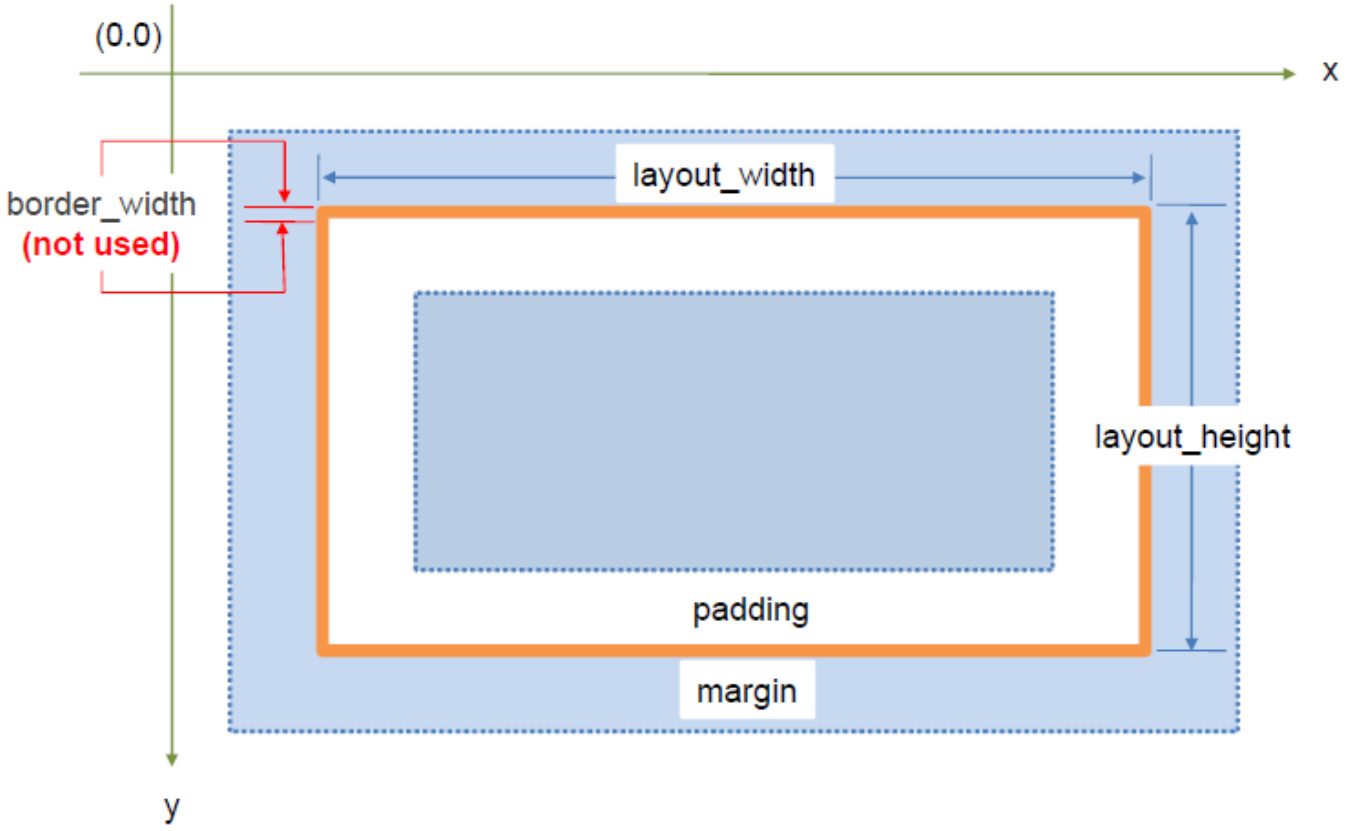
- Hierarchy View with parameters associated with each View



View: Attributes of Positioning



Attributes: Dimensions, Padding and Margins



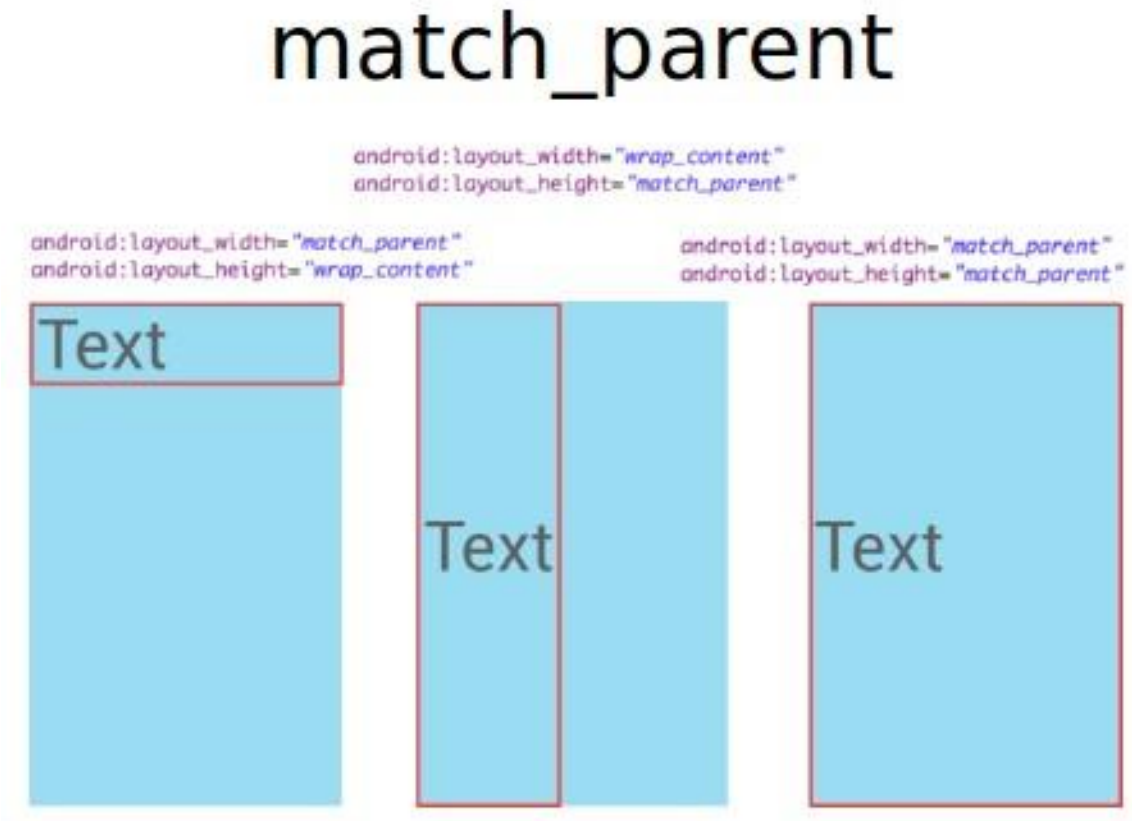
Attributes: most used ones

Atributos	Descrição
android:**layout_width**	layout width
android:**layout_height**	layout height
android:**layout_marginTop**	Extra space at the top of the layout
android:**layout_marginBottom**	Extra space at the bottom of the layout
android:**layout_marginLeft**	Extra space on the left side of the layout
android:**layout_marginRight**	Extra space on the right side of the layout
android:**layout_gravity**	Defines how child Views are positioned
android:**layout_weight**	Sets when extra space is allocated to View
android:**layout_x**	Sets the x coordinate of the layout
android:**layout_y**	Sets the y coordinate of the layout
android:**paddingLeft**	Padding on the left side of the layout
android:**paddingRight**	Padding on the right side of the layout
android:**paddingTop**	Padding on the top of the layout
android:**paddingBottom**	Padding at the bottom of the layout

android:layout_width and android:layout_height

- “width” and “height”
  - Units in "dp" (density-independent pixels)
- wrap_content
  - The dimensions of the View are set to the values needed for presenter content
- match_parent
  - The dimensions of the View increase to the dimensions that the parent ViewGroup allows

match_parent and wrap_content



Most used Layouts

Layout (ViewGroups)	Description
ConstraintLayout	Allows to create large and complex layouts with a flat view hierarchy (no nested view groups).
LinearLayout	Displays child elements in **a single horizontal or vertical line**. Creates a **scrollbar** if the window length exceeds the screen length
TableLayout	Group Views in **rows** and **columns**
FrameLayout	Allows to lock an area of the screen to show a single element
ListView	Displays a **vertically-scrollable collection of** views (legacy)
GridView	Shows a grid of elements with "**m**" columns and "**n**" **rows (legacy)**

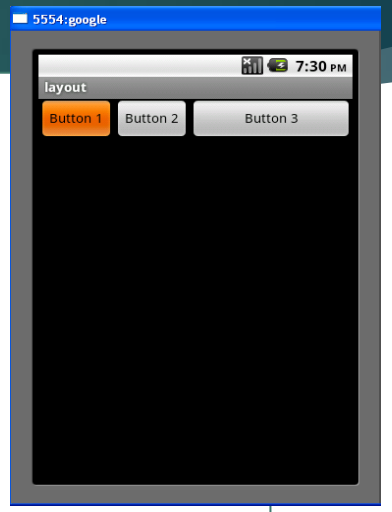
LinearLayout: horizontal/Vertical

```
<?xml version="1.0" encoding="“utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="horizontal" >

    <Button android:id="@+id/button1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 1" />

    <Button android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 2" />

    <Button android:id="@+id/button3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button 3"
        android:layout_weight="1"/>
</LinearLayout>
```



Layout_weight – element importance in occupied space (default value 0)

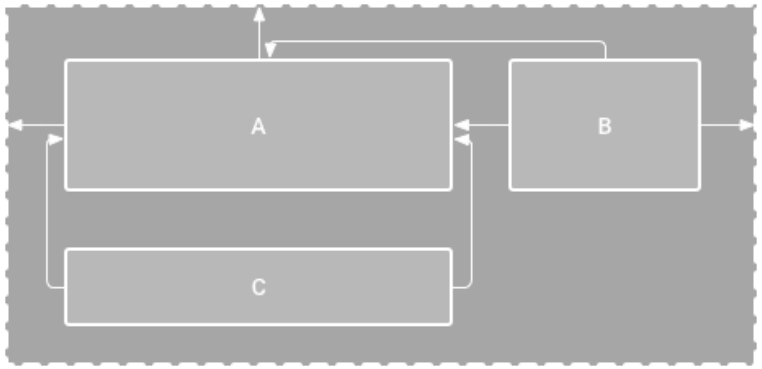
ConstraintLayout

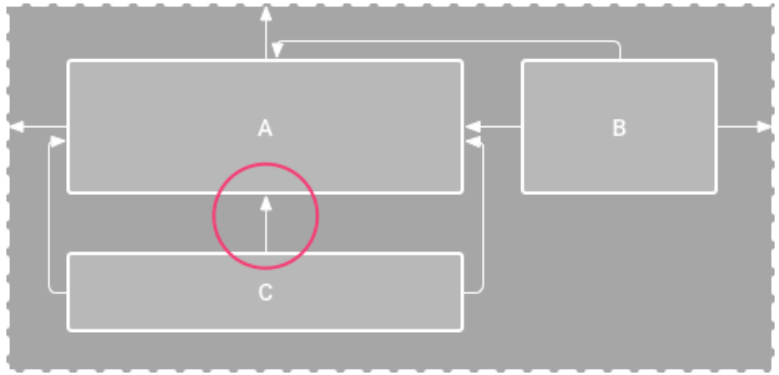
- To create complex layouts with a flat view hierarchy
  - no nested **ViewGroups** – at their top level
  - all views are laid out according to relationships between sibling and parent views
  - Similar to RelativeLayout but more **flexible** and **easier** to use with Android Studio's Layout Editor

https://developer.android.com/training/constraint-layout

ConstraintLayout: Example

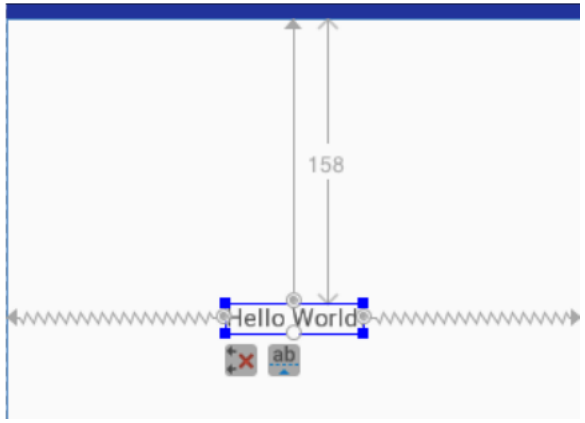
- On left figure no vertical constraint on view C
- On a device, view C horizontally aligns with the left and right edges of view A
- However, it appears at the top of the screen because it has no vertical constraint





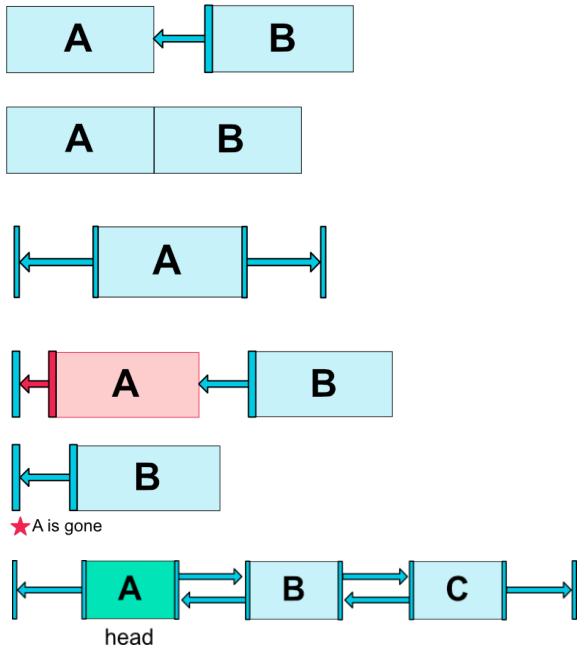
ConstraintLayout: Rules

- Every **View** must have at least two constraints: one **horizontal** and one **vertical**
- Constraints can only be created between a **constraint handle** and an **anchor point** that share the **same plane**
- **Constraint handle** can be used for just one constraint, but you can create multiple constraints to the same **anchor point**



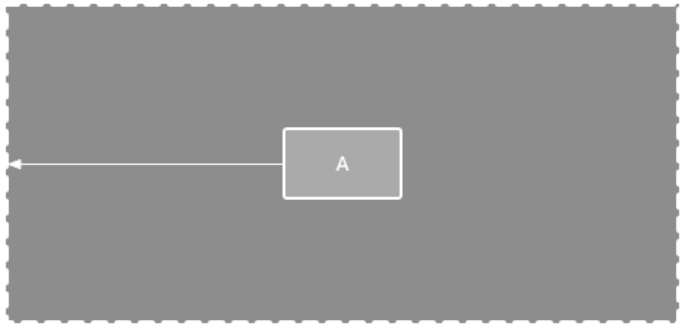
ConstraintLayout: Types of Constraints

- Relative Positioning
- Margins
- Centering Positioning
- Visibility Behavior
- Dimension Constraints
- Chains
- Virtual Helpers Objects (Guideline and Barrier)

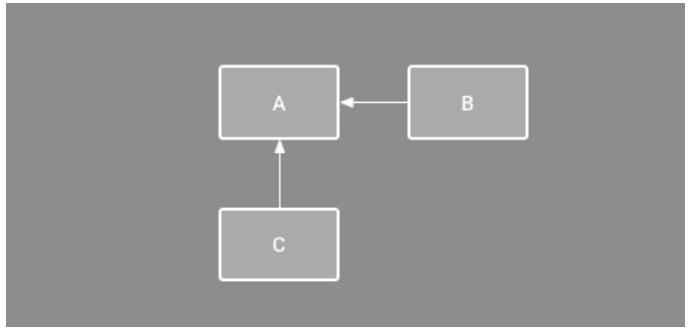


ConstraintLayout: Parent Position and Order Position

- Horizontal constraint to the parent

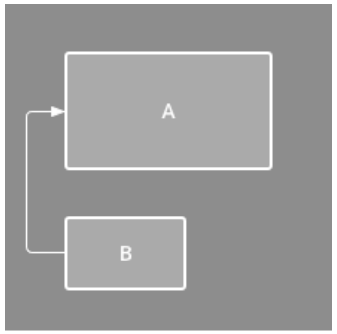


- Horizontal and vertical constraint

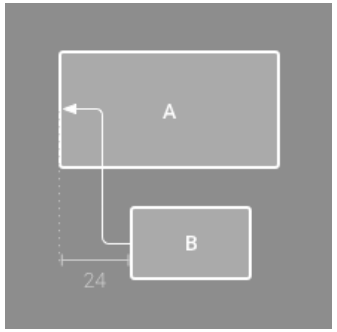


ConstraintLayout: Alignment

- Horizontal alignment constraint

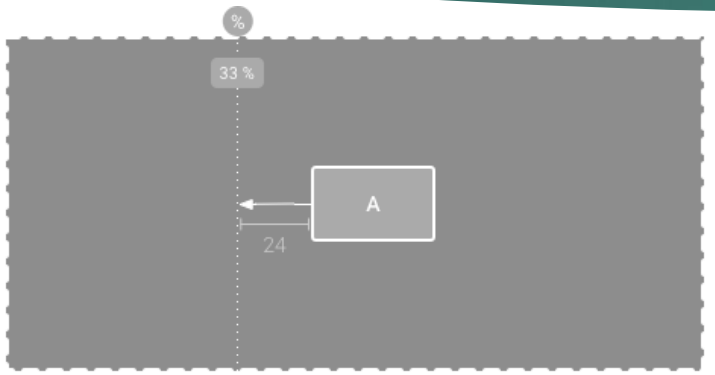


- An offset Horizontal alignment constraint

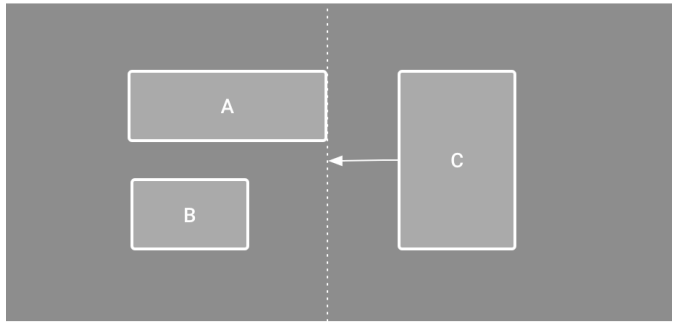


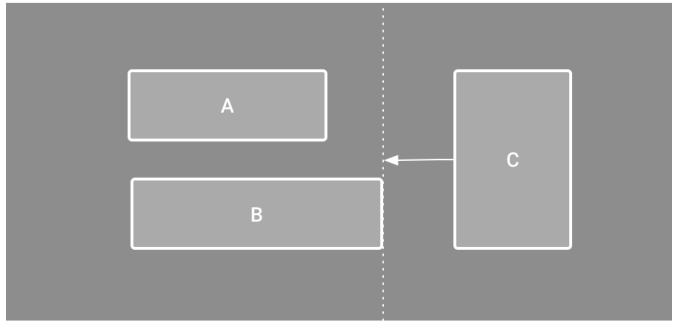
Click on the **align** icon in the toolbar

ConstraintLayout: Guideline and Barrier



View A is constrained to a **guideline**

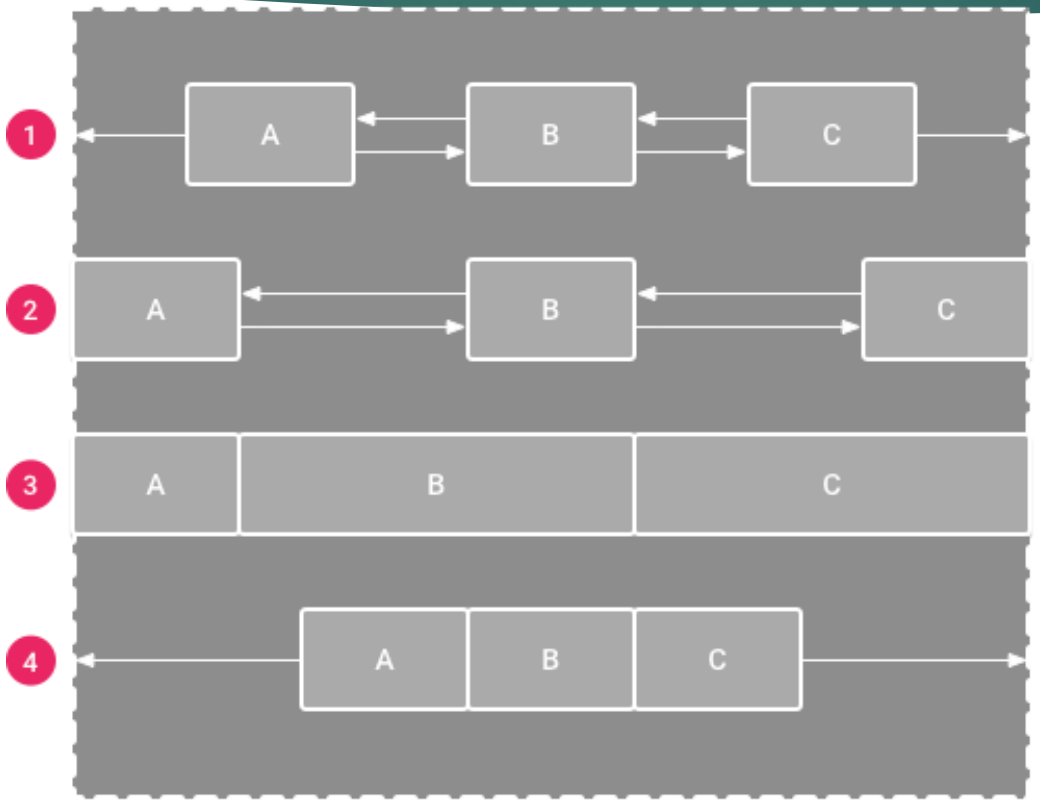




View C is constrained to a **barrier**, which moves based on the position/size of both view A and view B

Click on the **Guidelines** icon in the toolbar

ConstraintLayout: Chains



1- Spread

2- Spread inside

3- Weighted *

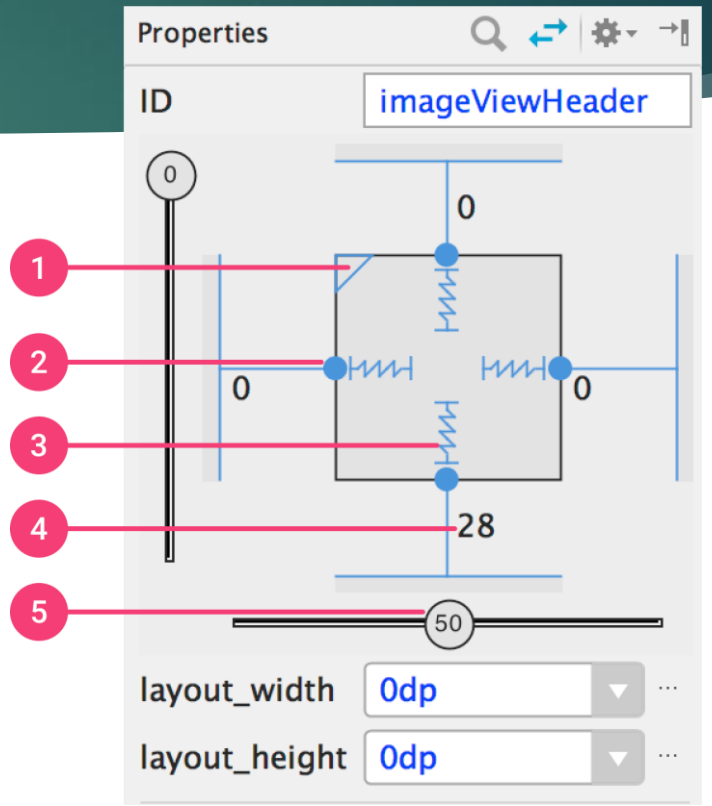
4- Packed

Click on the **Chain** button to change the style of the chain

* Views with match constraints option and using the layout_constraintHorizontal_weight property

ConstraintLayout: View Size

- wrap_content
- match_constraints
- fixed

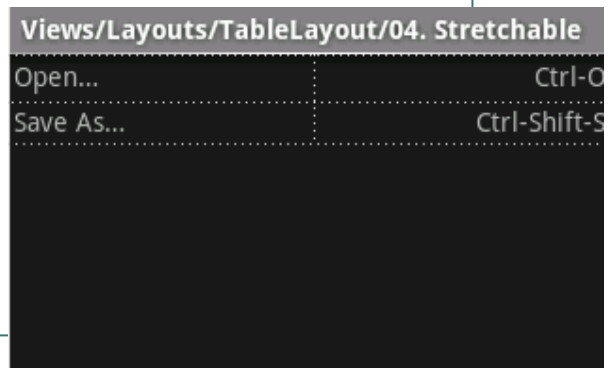


Do no use match_parent with ConstraintLayout.
Use match_contraints instead.

1 - size ratio, 2 - delete constraint,
3 - height/width mode, 4 - margins,
5 - constraint bias

TableLayout

```xml
<?xml version="1.0" encoding="“utf-8"?>
<TableLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:stretchColumns="1">
   <TableRow>
      <TextView
         android:text="@string/table_layout_4_open"
         android:padding="3dip" />
      <TextView
         android:text="@string/table_layout_4_open_shortcut"
         android:gravity="right"
         android:padding="3dip" />
   </TableRow>
   <TableRow>
      <TextView
         android:text="@string/table_layout_4_save"
         android:padding="3dip" />
      <TextView
         android:text="@string/table_layout_4_save_shortcut"
         android:gravity="right"
         android:padding="3dip" />
   </TableRow>
</TableLayout>
```

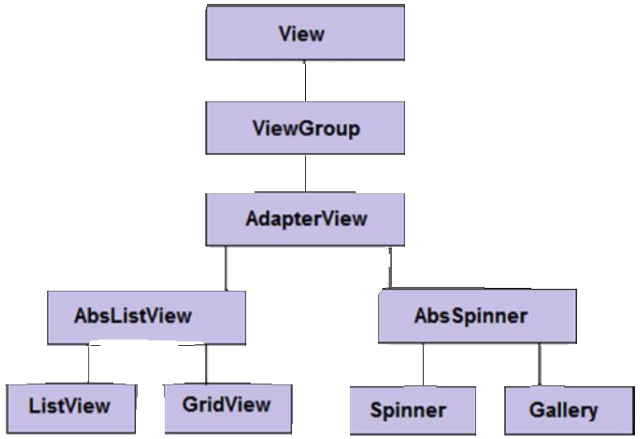


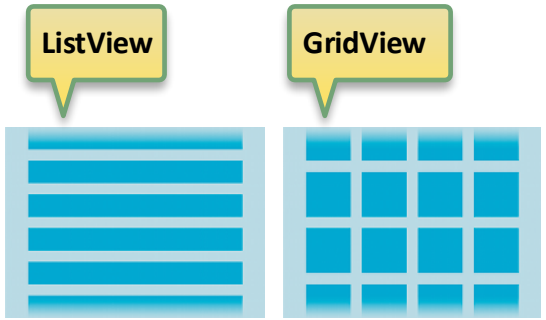
FrameLayout

- FrameLayout is designed to block out an area on the screen to display a single item
- In general, it should only contain a View because it is difficult to organize them so that they can be scalable to any screen without overlapping
- To have more than one View one must use the attribute "gravity" in each View "child" (android:layout_gravity)

ListView and GridView

- Dynamic Content
  - **AdapterView** subclass to put values in Layout
  - Class Adapter to fetch the data (e.g., from a database) and convert them into Views that are added to the **AdapterView**





Widgets: Android UI Controls

- TextView
- EditText
- Button
- Checkbox
- RadioButton
- ToggleButton / Switch
- Spinner
- ProgressBar
- TimePicker

UI Controls

UI Controls	Description
TextView	Used to show text to user
EditText	Pre-defined **TextView** sub-class which includes the possibility of editing the text
AutoCompleteTextView	Similar to **EditText** but shows a list with suggestions to help complete the text
Button / ImageButton	**Button** that can be pressed by the user / Button with an Image
CheckBox	To select options that are not **mutually exclusive**
RadioButton / RadioGroup	To select options that are **mutually exclusive**. To have that, they must be **registered in RadioGroup**
ToggleButton / Switch	A button that has two states (ex: on / off)
Spinner	Provides a way to select **one value** from a list or a set
ProgressBar	Provides **visual feedback** about a task that is being performed
Picker	**TimePicker** or **DatePicker**. Allows users to choose a time of day or a date

Android Studio Pallete groups

Text

- TextView
- Plain Text
- Password
- Password (Numeric)
- E-mail
- Phone
- Postal Address
- Multiline Text
- Time
- Date
- Number
- Number (Signed)
- Number (Decimal)
- AutoCompleteTextView
- MultiAutoCompleteText...
- CheckedTextView
- TextInputLayout

Buttons

- Button
- ImageButton
- ChipGroup
- Chip
- CheckBox
- RadioGroup
- RadioButton
- ToggleButton
- Switch
- FloatingActionButton

Widgets

- View
- ImageView
- WebView
- VideoView
- CalendarView
- ProgressBar
- ProgressBar (Horizontal)
- SeekBar
- SeekBar (Discrete)
- RatingBar
- SearchView
- TextureView
- SurfaceView
- Horizontal Divider
- Vertical Divider

Layouts

- ConstraintLayout
- Guideline (Horizontal)
- Guideline (Vertical)
- LinearLayout (horizontal)
- LinearLayout (vertical)
- FrameLayout
- TableLayout
- TableRow
- Space

Containers

- Spinner
- RecyclerView
- ScrollView
- HorizontalScrollView
- NestedScrollView
- ViewPager
- CardView
- AppBarLayout
- BottomAppBar
- NavigationView
- BottomNavigationView
- Toolbar
- TabLayout
- TabItem
- ViewStub
- <include>
- <fragment>
- NavHostFragment
- <view>
- <requestFocus>

UI with dynamic changes in code

- Building UI Elements - declared in XML

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

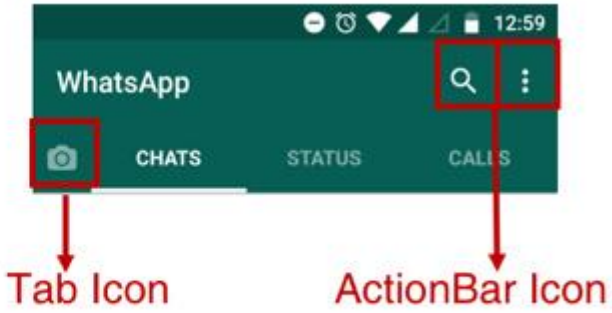
- Building UI Elements - Created in code - In code we can create any element and set any attribute to them

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    val constrLayout = ConstraintLayout(this)
    val tv = TextView(this)
    tv.text = "Test"
    constrLayout.addView(tv)
    setContentView(constrLayout)
}
```

Icons & Images

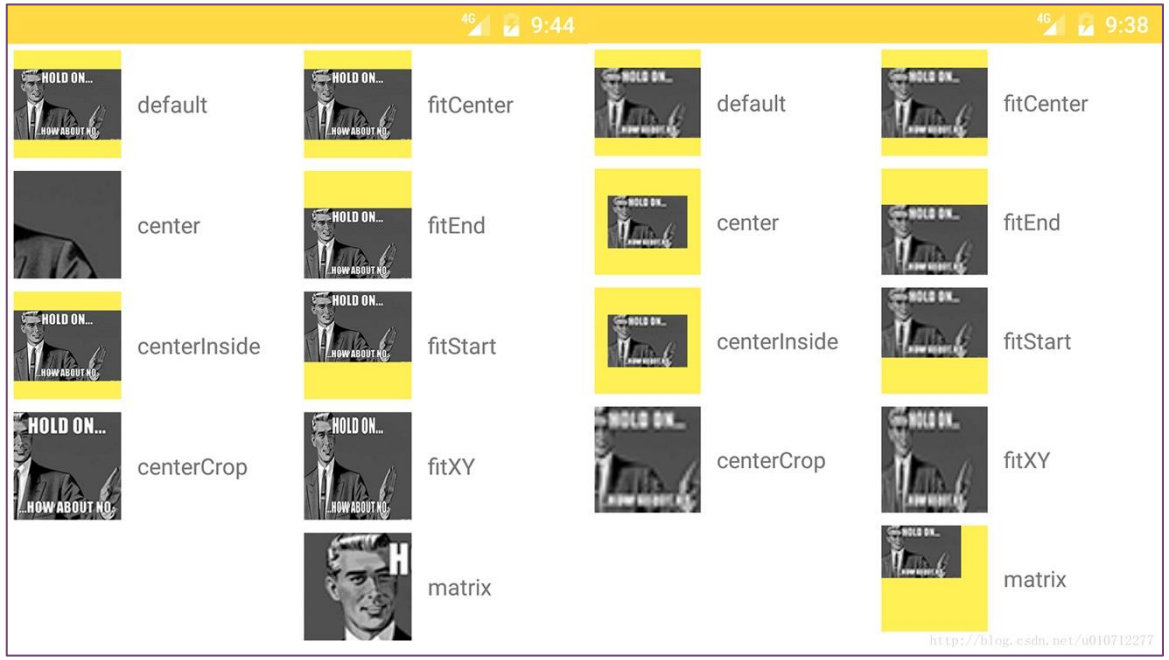
Icons

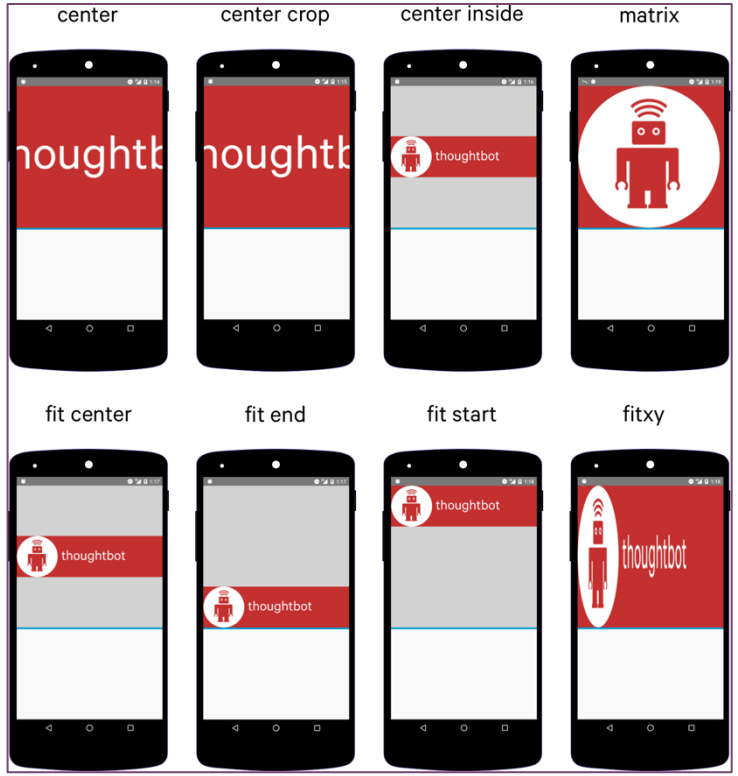
- Icon categories:
  - **Launcher Icon**
    - Adaptive Launcher Icon, for Android 8.0 (API level 26) or above
    - Legacy Launcher Icon, for Android 7.1 (API level 25) or below.
  - **Action Bar Icons**: for the items in the action bar
  - **Tab Icons:** for the tab items that have icons, if any



https://code.tutsplus.com/tutorials/how-to-update-the-launcher-icon-for-your-android-app--cms-30624

Images: ScaleType





https://blog.csdn.net/u010712277/article/details/74079620
https://thoughtbot.com/blog/android-imageview-scaletype-a-visual-guide

Bibliography

- Layouts
- Input Controls
- Layout Resources (attributes)
- ConstraintLayout
- Performance and View Hierarchies